```
from google.colab import files
uploaded = files.upload()
```

Choose Files 3 files

Customer.xls(application/vnd.ms-excel) - 31232 bytes, last modified: n/a - 100% done
Order.csv(text/csv) - 5221 bytes, last modified: n/a - 100% done
Shipping.json(application/json) - 20230 bytes, last modified: n/a - 100% done

```
Saving Customer.xls to Customer (1).xls
Saving Order.csv to Order.csv
Saving Shipping.json to Shipping.json
```

```
import pandas as pd

orders = pd.read_csv("Order.csv")
customers = pd.read_excel("Customer.xls")
shipping = pd.read_json("Shipping.json")
```

```
import sqlite3

conn = sqlite3.connect(":memory:")

orders.to_sql("orders", conn, index=False)
customers.to_sql("customers", conn, index=False)
shipping.to_sql("shipping", conn, index=False)
```

```
250
```

```
import pandas as pd

orders = pd.read_csv("Order.csv")
customers = pd.read_excel("Customer.xls")
shipping = pd.read_json("Shipping.json")

print("Orders shape:", orders.shape)
print("Customers shape:", customers.shape)
print("Shipping shape:", shipping.shape)
```

```
Orders shape: (250, 4)
Customers shape: (250, 5)
Shipping shape: (250, 3)
```

```
orders.head()
```

	Order_ID	Item	Amount	Customer_ID
0	1	Keyboard	400	139
1	2	Mouse	300	250
2	3	Monitor	12000	239
3	4	Keyboard	400	153
4	5	Mousepad	250	153

Next steps: Generate code with orders New interactive sheet

```
customers.head()
```

	Customer_ID	First	Last	Age	Country
0	1	Joseph	Rice	43	USA
1	2	Gary	Moore	71	USA
2	3	John	Walker	44	UK
3	4	Eric	Carter	38	UK
4	5	William	Jackson	58	UAE

Next steps: Generate code with customers New interactive sheet

```
shipping.head()
```

	Shipping_ID	Status	Customer_ID
0	1	Pending	173
1	2	Pending	155
2	3	Delivered	242
3	4	Pending	223
4	5	Delivered	72

Next steps: Generate code with shipping New interactive sheet

Data Validation

In this section, we verify:

- Missing values
- Duplicate records
- Referential integrity
- Data consistency

```
orders.isnull().sum()
customers.isnull().sum()
shipping.isnull().sum()
```

	0
Shipping_ID	0
Status	0
Customer_ID	0

dtype: int64

```
orders.duplicated().sum()
customers.duplicated().sum()
shipping.duplicated().sum()
```

```
np.int64(0)
```

```
missing_customers = orders[~orders['Customer_ID'].isin(customers['C
missing_customers.shape
```

```
(0, 4)
```

```
shipping['Status'].unique()
```

```
array(['Pending', 'Delivered'], dtype=object)
```

```
import sqlite3

conn = sqlite3.connect(":memory:")

orders.to_sql("orders", conn, index=False, if_exists='replace')
customers.to_sql("customers", conn, index=False, if_exists='replace
shipping.to_sql("shipping", conn, index=False, if_exists='replace')
```

```
250
```

```
customers.columns
```

```
Index(['Customer_ID', 'First', 'Last', 'Age', 'Country'],
dtype='object')
```

```
orders.columns
```

```
Index(['Order_ID', 'Item', 'Amount', 'Customer_ID'],
dtype='object')
```

```
query1 = """
SELECT
    c.Country,
    SUM(o.Amount) AS total_spent
FROM orders o
JOIN customers c ON o.Customer_ID = c.Customer_ID
JOIN shipping s ON s.Customer_ID = c.Customer_ID
WHERE s.Status = 'Pending'
GROUP BY c.Country
ORDER BY total_spent DESC
"""
result1 = pd.read_sql(query1, conn)
result1
```

1 to 3 of 3 entries Filter

index	Country	total_spent
0	UK	136300
1	USA	65500
2	UAE	53800

Show 25 per page

Like what you see? Visit the data table notebook to learn more about interactive tables.

Next steps: Generate code with result1 New interactive sheet

```
query2 = """
SELECT
    c.Customer_ID,
    c.First || ' ' || c.Last AS Full_Name,
    COUNT(o.Order_ID) AS total_transactions,
    SUM(o.Amount) AS total_spent
FROM orders o
JOIN customers c ON o.Customer_ID = c.Customer_ID
GROUP BY c.Customer_ID, Full_Name
ORDER BY total_spent DESC
"""
result2 = pd.read_sql(query2, conn)
result2.head()
```

1 to 5 of 5 entries Filter

index	Customer_ID	Full_Name	total_transactions	total_spent
0	166	Morgan Cooper	3	17350
1	123	Philip Robinson	2	17000
2	129	Amber Banks	2	17000
3	96	Eric Harvey	4	14700
4	193	Yesenia White	4	13950

Show 25 per page

Like what you see? Visit the data table notebook to learn more about interactive tables.

Next steps: Generate code with result2 New interactive sheet

```
query3 = """
SELECT Country, Item, total_orders
FROM (
    SELECT
        c.Country,
        o.Item,
        COUNT(o.Order_ID) AS total_orders,
        RANK() OVER (
            PARTITION BY c.Country
            ORDER BY COUNT(o.Order_ID) DESC
        ) rnk
    FROM orders o
    JOIN customers c ON o.Customer_ID = c.Customer_ID
    GROUP BY c.Country, o.Item
)
WHERE rnk = 1
"""
result3 = pd.read_sql(query3, conn)
result3
```

	Country	Item	total_orders
0	UAE	Keyboard	12
1	UK	Mousepad	24
2	USA	Mousepad	18

Next steps: Generate code with result3 New interactive sheet

```
query4 = """
SELECT age_group, Item, total_orders
FROM (
    SELECT
        CASE
            WHEN c.Age < 30 THEN 'Below 30'
            ELSE 'Above 30'
        END AS age_group,
        o.Item,
        COUNT(o.Order_ID) AS total_orders,
        RANK() OVER (
            PARTITION BY
                CASE WHEN c.Age < 30 THEN 'Below 30' ELSE 'Above 30
            ORDER BY COUNT(o.Order_ID) DESC
        ) rnk
    FROM orders o
    JOIN customers c ON o.Customer_ID = c.Customer_ID
    GROUP BY age_group, o.Item
)
WHERE rnk = 1
"""
result4 = pd.read_sql(query4, conn)
result4
```

	age_group	Item	total_orders
0	Above 30	Keyboard	35
1	Below 30	Mousepad	17

Next steps: Generate code with result4 New interactive sheet

```
query5 = """
SELECT Country, transactions, sales
FROM (
    SELECT
        c.Country,
        COUNT(o.Order_ID) AS transactions,
        SUM(o.Amount) AS sales,
        RANK() OVER (
            ORDER BY COUNT(o.Order_ID), SUM(o.Amount)
        ) rnk
    FROM orders o
    JOIN customers c ON o.Customer_ID = c.Customer_ID
    GROUP BY c.Country
)
WHERE rnk = 1
"""
result5 = pd.read_sql(query5, conn)
result5
```

	Country	transactions	sales
0	UAE	40	49950

```
query6 = """
SELECT
    Status,
    COUNT(*) * 100.0 / (SELECT COUNT(*) FROM shipping) AS percentag
FROM shipping
GROUP BY Status
"""
result6 = pd.read_sql(query6, conn)
result6
```

1 to 2 of 2 entries Filter

index	Status	percentage
0	Delivered	40.0
1	Pending	60.0

Show 25 per page

Like what you see? Visit the data table notebook to learn more about interactive tables.

Next steps: Generate code with result6 New interactive sheet

Data Understanding

- The dataset does not contain quantity or product ID
- `Item` represents the product
- Each row represents one transaction

Adjustments Made

- Used COUNT(Order_ID) instead of quantity
- Used `Item` instead of product_id

Data Model Design

Entities Identified

1. Customers
    - Customer_ID (Primary Key)
    - First, Last, Age, Country
2. Orders
    - Order_ID (Primary Key)
    - Customer_ID (Foreign Key)
    - Item
    - Amount
3. Shipping
    - Shipping_ID
    - Customer_ID (Foreign Key)
    - Status

Relationships

- One Customer → Many Orders
- One Customer → Many Shipping records

Assumptions

- Each order represents one item purchase
- Shipping is tracked at customer level

Data Engineering Story: Customer Sales Summary Table

Objective

Create a table that aggregates customer-level metrics for reporting.

Table Name

customer_sales_summary

Columns

- Customer_ID
- Full_Name
- Total_Transactions
- Total_Spent

Transformations

- Join Orders and Customers on Customer_ID
- Create Full_Name using First + Last
- Count total transactions using Order_ID
- Sum total spending using Amount

SQL Logic

SELECT c.Customer_ID, c.First || ' ' || c.Last AS Full_Name, COUNT(o.Order_ID) AS Total_Transactions, SUM(o.Amount) AS Total_Spent FROM orders o JOIN customers c ON o.Customer_ID = c.Customer_ID GROUP BY c.Customer_ID

QA Checks

- Ensure no duplicate Customer_ID
- Validate total_spent > 0
- Cross-check transaction counts with raw data

```
Start coding or generate with AI.
```

Insight: Shipping Performance

- 60% of orders are still in **Pending** status
- Only 40% of orders are **Delivered**

Business Interpretation

This indicates a potential issue in the delivery pipeline:

- High backlog of pending shipments
- Possible delays in logistics or fulfillment
- Risk of poor customer experience

Recommendation

- Investigate bottlenecks in shipping operations
- Prioritize clearing pending orders
- Monitor delivery SLAs more closely

```
Start coding or generate with AI.
```